UNIVERSIDAD NACIONAL DE CÓRDOBA
FACULTAD DE CIENCIAS EXACTAS FÍSICAS Y NATURALES

PROCESAMIENTO DIGITAL DE SEÑALES

LABORATORIO 1: Muestreo

Profesores:

Parlanti, Gustavo.
Molina, Germán.
Rossi, Roberto.

Alumnos:

Choque, Fernando Rafael Bernardo.
Corvalan, Abel.
Reyeros, Marcos.

TABLA DE CONTENIDO

1. PROYECTO
2. DESCRIPCIÓN DEL SEP (Sistema Electrónico Programable)
3. DESARROLLO DE HARDWARE
4. DESARROLLO DE FIRMWARE
5. PRUEBAS DE SISTEMA
6. CONCLUSIONES
7. BIBLIOGRAFÍA Y REFERENCIAS
8. ANEXO

1 . PROYECTO:

El presente Laboratorio tiene por objetivo desarrollar un sistema de adquisición y reconstrucción de señales analógicas utilizando los módulos periféricos del microcontrolador MCXN947VDF, integrado en la placa de desarrollo FRDM - MCXN947 de NXP.

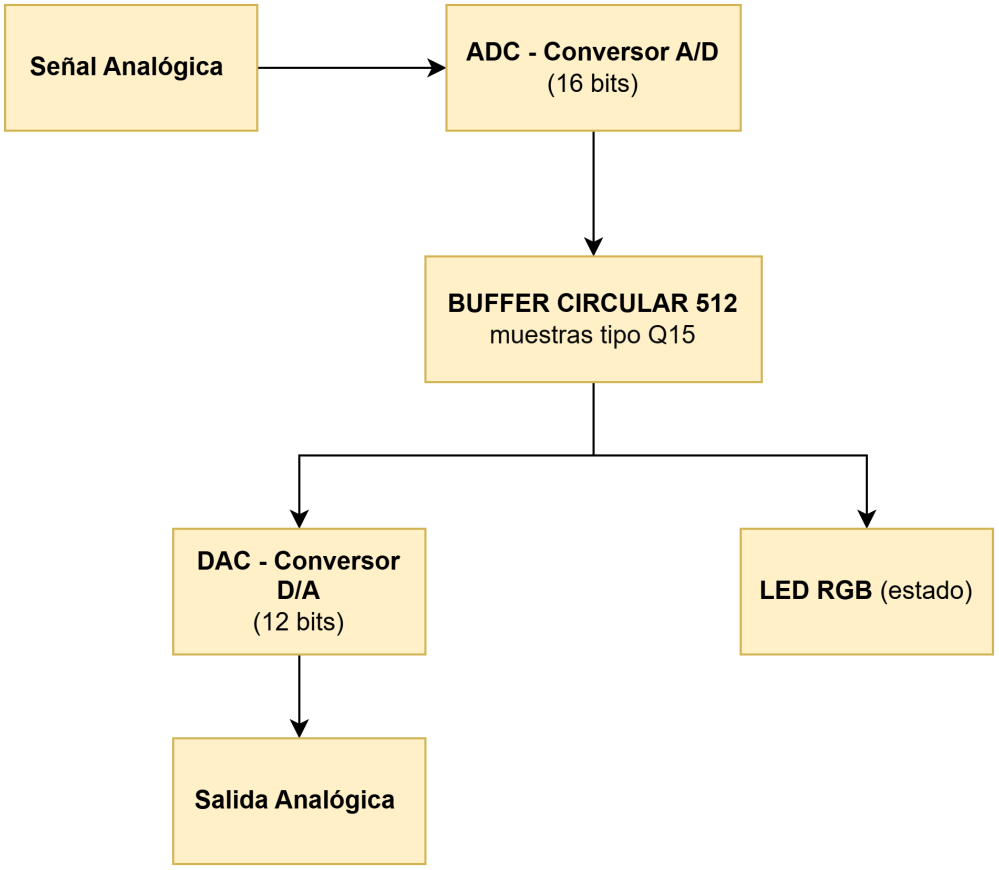
El sistema permite digitalizar una señal analógica mediante el módulo ADC (Analog-to-Digital Converter) del microcontrolador a distintas velocidades de muestreo, seleccionables por el usuario. Las velocidades implementadas son 8 kS/s, 16 kS/s, 22 kS/s, 44 kS/s y 48 kS/s.

Cada cambio de frecuencia se realiza a través de una de las teclas de usuario incorporadas en la placa (SW1), funcionando como un selector circular de modos. El color del LED RGB integrado indica visualmente la frecuencia seleccionada.

Una segunda tecla (SW2) permite habilitar o detener la adquisición de datos (modo Run/Stop). Los valores adquiridos se almacenan en un buffer circular de 512 muestras, en formato q15 (fraccional de 15 bits), lo que permite mantener una estructura continua de datos sin pérdida de información.

En simultáneo, los valores digitalizados son enviados al DAC (Digital-to-Analog Converter) de 12 bits, que reconstruye la señal analógica a la salida, posibilitando observar la respuesta del sistema y verificar la fidelidad de la conversión.

2. DESCRIPCIÓN DEL SEP:



El **Sistema Embebido Programable (SEP)** desarrollado tiene como finalidad la **adquisición, almacenamiento y reproducción digital de señales analógicas** a distintas tasas de muestreo, utilizando los recursos periféricos integrados del microcontrolador **MCXN947**.

El sistema está compuesto por los siguientes bloques funcionales:

1. **Módulo de Adquisición (ADC):**
   Captura la señal analógica proveniente de un canal configurado del conversor A/D. El ADC es controlado mediante interrupciones generadas por un temporizador (TIMER 0 - Match3) que define la frecuencia de muestreo.
   La señal analógica se convierte a digital con una resolución de 16 bits y se almacena en el buffer circular.

2. **Buffer Circular de Datos:**
   Implementa una estructura de 512 posiciones para almacenar las muestras q15 obtenidas.
   El buffer actúa como un FIFO circular, evitando desbordes y permitiendo la actualización continua de los datos.
   Se utiliza para la transferencia tanto hacia el DAC y almacenamiento en formato q15

3. **Módulo DAC (Conversor Digital-Analógico):**
   Convierte las muestras digitales almacenadas en el buffer nuevamente a una señal analógica, pero en este caso pasamos directo la señal digitalizada al DAC.
   Su resolución de 12 bits permite evaluar la respuesta temporal del sistema frente a

distintas tasas de muestreo, se debe hacer un escaleo dado que el ADC nos dio muestras de 16 bits

4. **Interfaz de Usuario:**
    - Tecla SW2: cambia la frecuencia de muestreo de forma circular entre los cinco valores disponibles.
    - Tecla SW3: inicia o detiene el proceso de adquisición (Run/Stop).
    - LED RGB: indica visualmente el estado actual de muestreo mediante un color característico para cada frecuencia.

El flujo general de señal dentro del sistema se resume en la siguiente secuencia:

1. La señal analógica es aplicada a la entrada del **ADC**.
2. El **temporizador** genera interrupciones periódicas a la frecuencia seleccionada.
3. En cada interrupción, el valor convertido se almacena en el **buffer circular** y se saca por el DAC.
4. El usuario puede interactuar en tiempo real modificando la frecuencia o deteniendo la adquisición.

4. DESARROLLO DE FIRMWARE

Configuraciones por interfaz (ConfigTools):

A continuación se muestran las configuraciones realizadas para los módulos periféricos implementados en el proyecto, estos son: CTIMER0, ADC (Conversor analógico-digital), DAC (Conversor digital-analógico), módulo de tensión de referencia (VREF), controlador de interrupciones vectorizadas anidadas (NVIC) y la habilitación de los relojes de los periféricos nombrados.

CTIMER0:

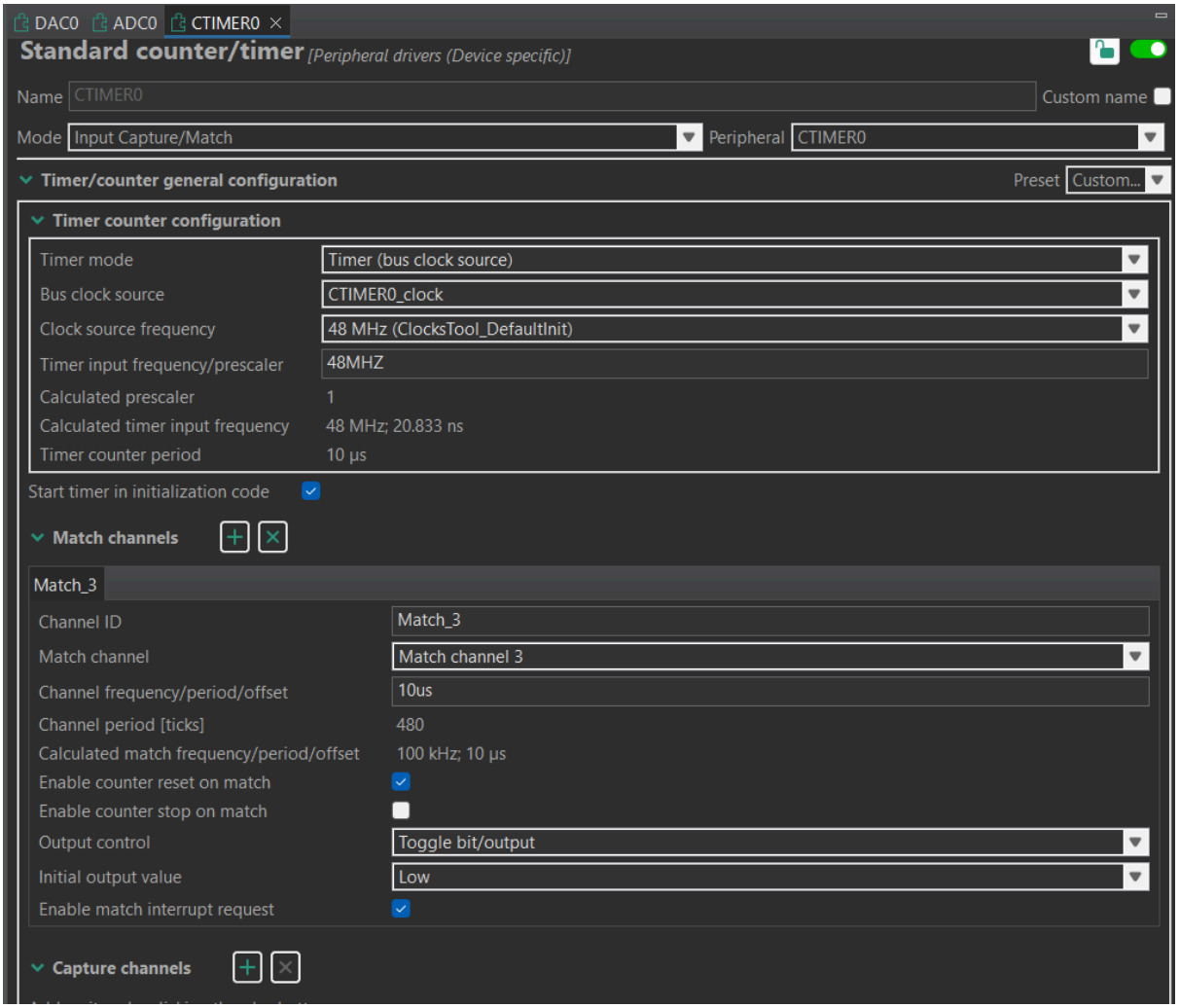
Se configura el timer con un clock de 48MHz, con el cual se obtiene un tiempo de Ticks de 20.83ns.

$$T_{tick} = \frac{1}{48MHz} = 20{,}83ns$$

Se configura con un periodo de cuenta de 10us. Por lo que se tiene:

$$T_{10\mu s} = \frac{10\mu s}{20{,}83ns} = 480ticks$$

Se configura el pin de salida para obtener cambios de estados entre 1 y 0 lógicos. El valor inicial de este pin es cero lógico.



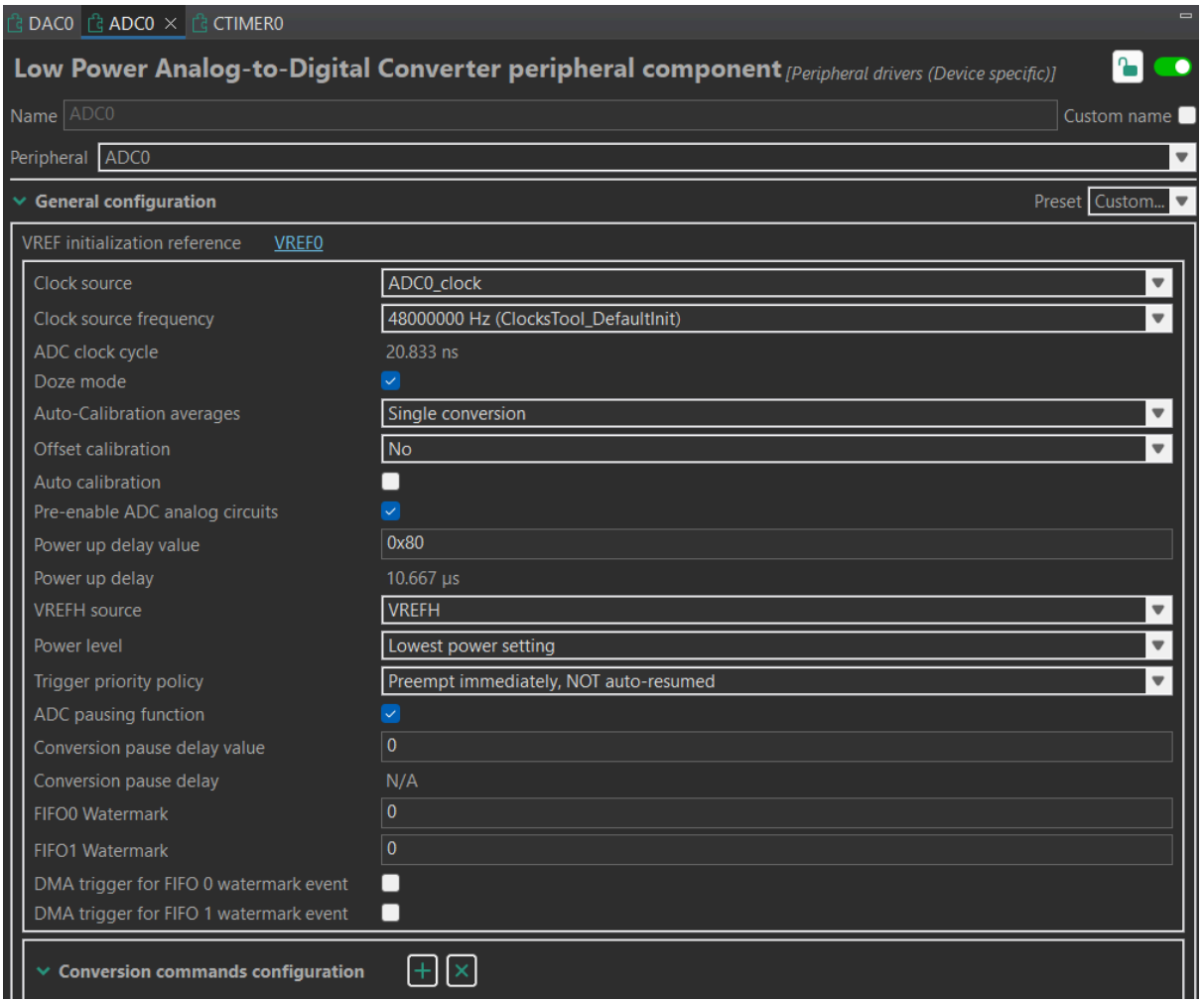
ADC:

El módulo ADC0 de la placa MCXN947 se configura con un clock de 48MHz. Se establece su funcionamiento en modo doze, el cual permite le permite continuar operativo mientras el CPU entra en régimen de bajo consumo. Para la calibración se basa en una conversión promedio (Auto-Calibration averages). Se habilita el circuito analógico del ADC antes de iniciar la conversión (reduce el tiempo de espera). Se configura un valor de retardo de 0x80 para estabilizar el ADC al momento de encenderlo. Se configura la tensión de referencia VREFH, en este caso 3.3v. Se configura el modo de consumo mínimo. Se ahorra energía sacrificando velocidad en la conversión.

Entre otras de las configuraciones se establece la mayor prioridad en la conversión al momento de la interrupción, esto significa, que en caso de estar realizando una conversión

sin haber sido completada y se tiene una interrupción se le da prioridad a la nueva conversión (no se reanuda la conversión anterior).

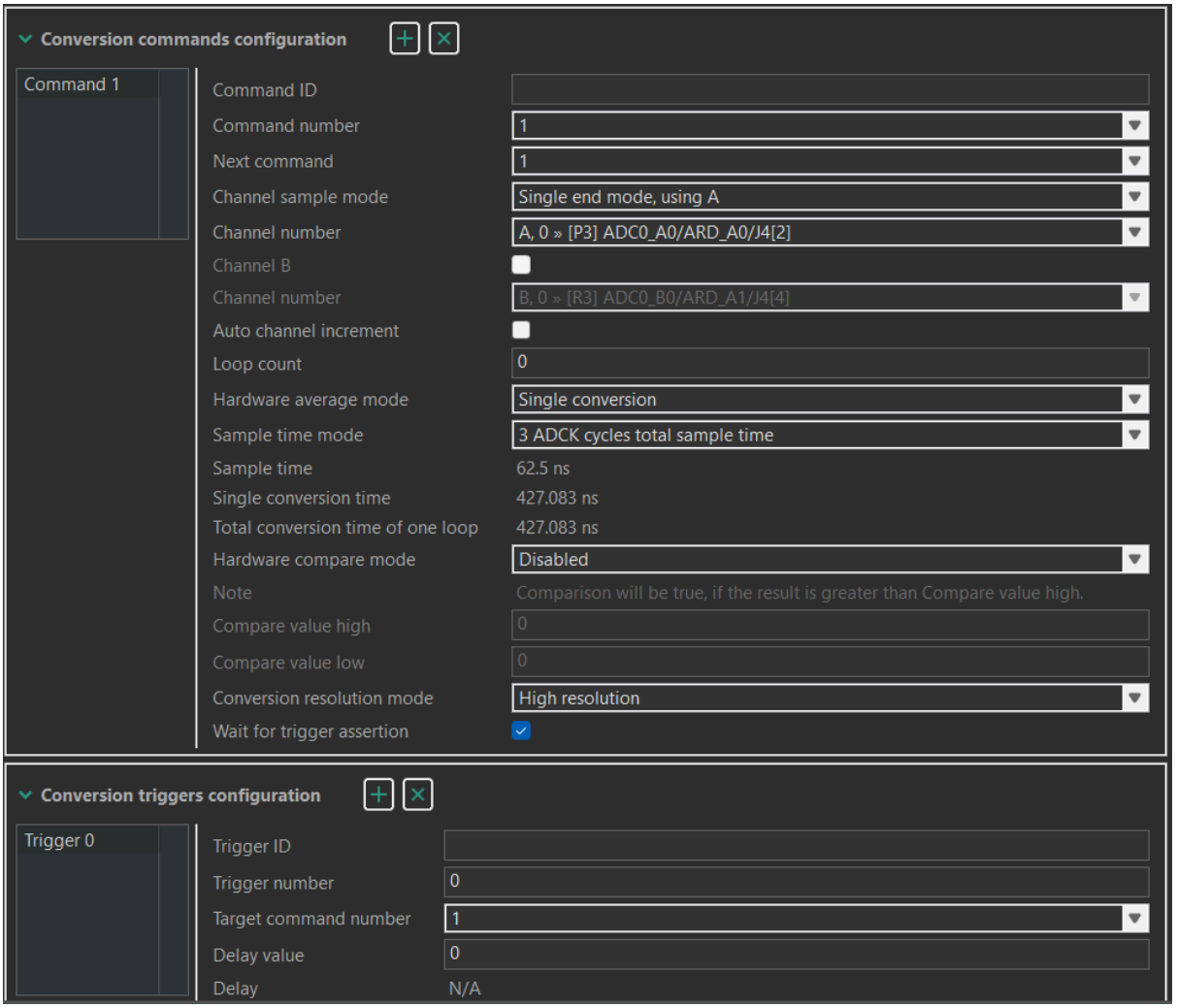
Se permite pausar la conversión si el software lo requiere. Se configura un retardo nulo entre conversiones del módulo. No se generan interrupciones por FIFO llena (no se implementa el nivel de umbral en los buffers FIFO). No se habilita el módulo DMA para transferir resultados.



Se determina el identificador del comando de conversión (comando 1). Se medirá una sola entrada analógica respecto a GND (Channel sample mode, single end mode).

Se configura el canal analógico A0. Se anula el canal B, ya que no se va a realizar una medición en modo diferencial. Con respecto al Loop count, se configura para que la conversión se realice una sola vez por trigger.

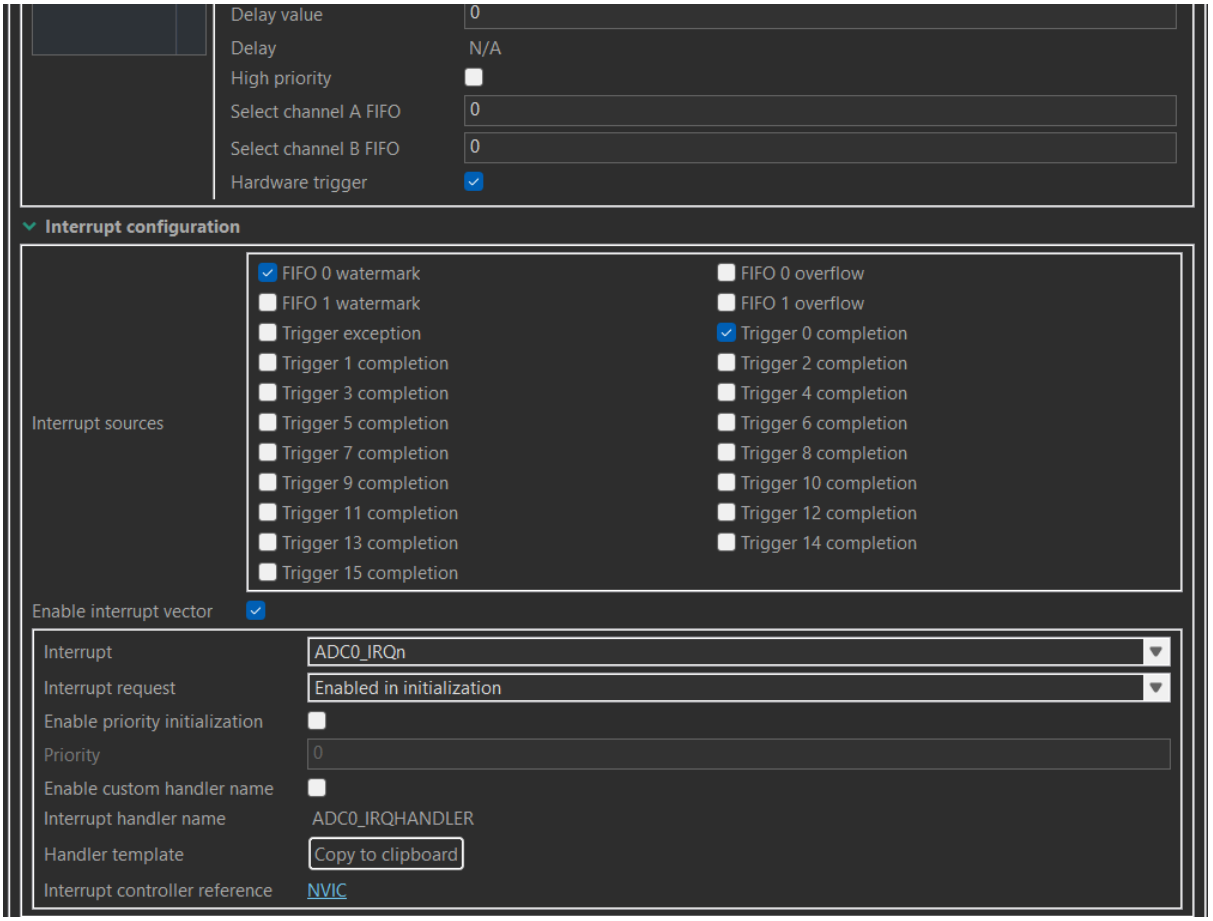
No se realiza el promedio entre varias lecturas, cada conversión entrega un único valor. El sample time mode se configura con el match del timer0. Además se configura el modo de alta resolución del ADC.



En esta configuración del ADC0 se observa la sección correspondiente a la parte de interrupciones y manejo de disparadores de hardware. El módulo está configurado para utilizar un hardware trigger, es decir, la conversión no se inicia por software sino por un evento externo o interno que actúa como disparador. Además, el disparador seleccionado es el Trigger 0.

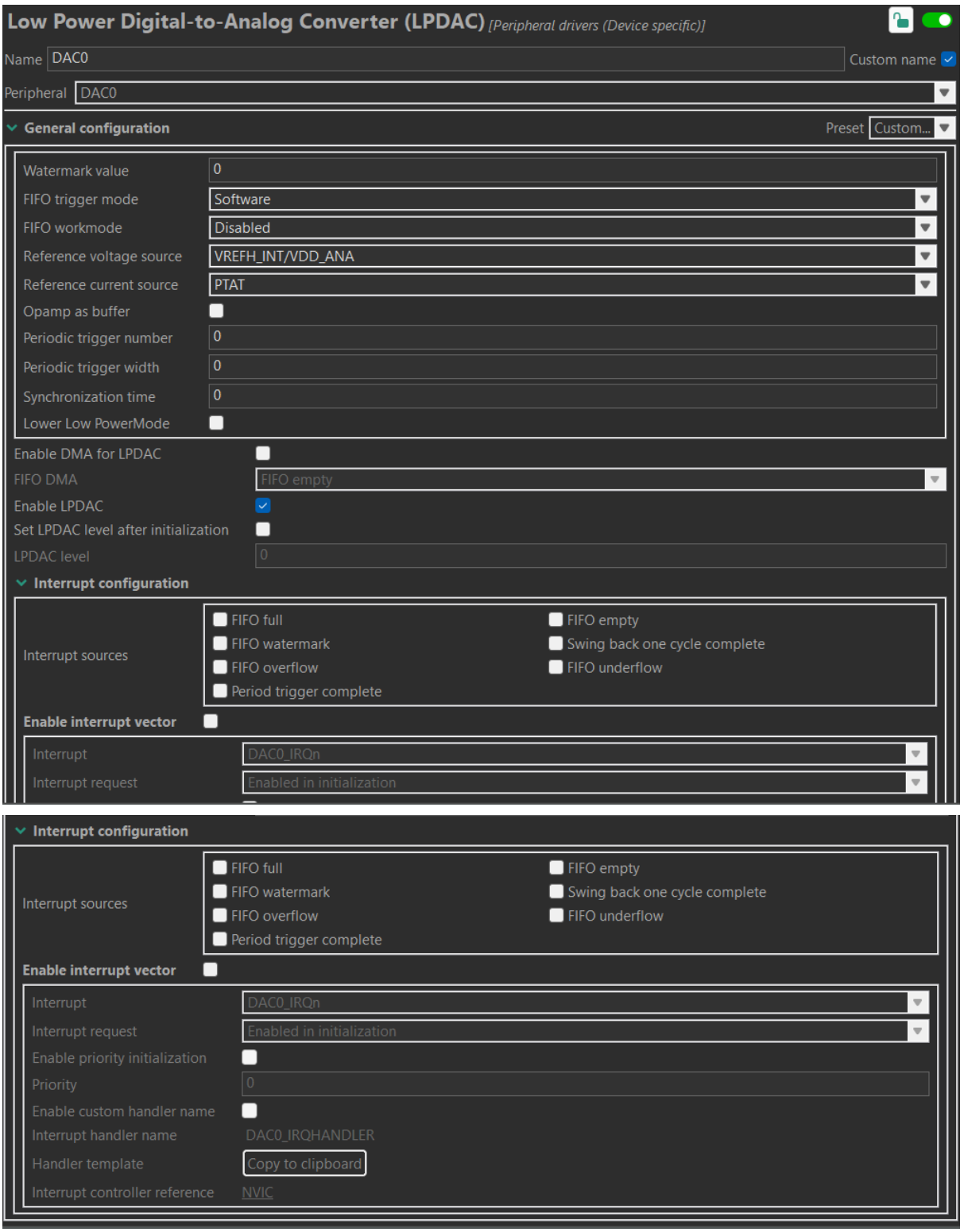
Dentro de la sección Interrupt configuration, se seleccionan las fuentes de interrupción que hacen que el microcontrolador ejecute una rutina específica cuando ocurra un evento dentro del ADC. En este caso, se han habilitado las interrupciones por FIFO 0 watermark y por Trigger 0 completion y Trigger 2 completion.

El vector de interrupción seleccionado es ADC0_IRQn, que corresponde al manejador de interrupciones del periférico ADC0 en el NVIC (Nested Vectored Interrupt Controller). Se define el nombre de la función de servicio como ADC0_IRQHandler, que es la rutina que se ejecutará cuando ocurra una interrupción proveniente del ADC.



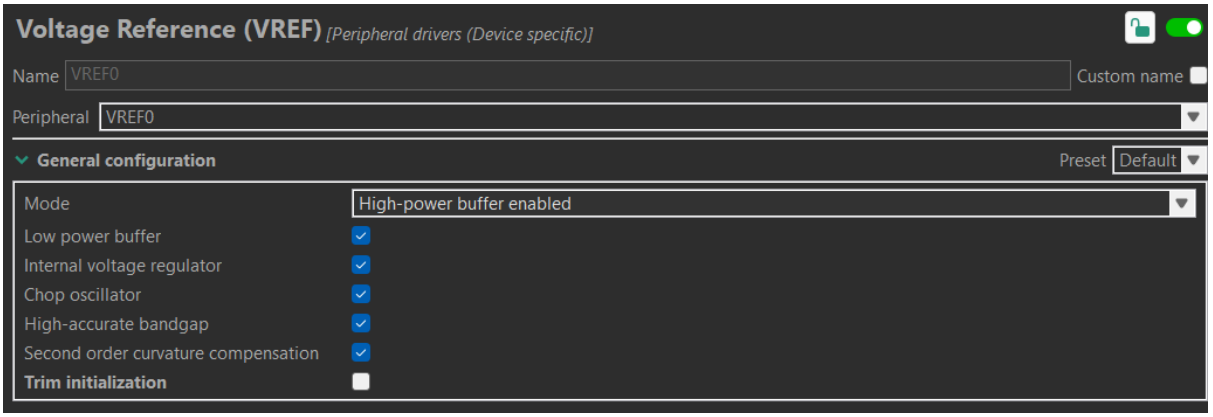
DAC0:

El DAC0 (Digital to Analog Converter) convierte valores digitales en señales analógicas. En esta configuración, trabaja en modo software, sin FIFO habilitado. Las actualizaciones se hacen manualmente desde el software. Usa como referencia de voltaje VREFH_INT/VDD_ANA. No se habilitan los modos periódicos ni sincronización, y opera fuera del modo de bajo consumo. Se activa el LPDAC (Low Power DAC) con un nivel inicial de salida igual a cero.



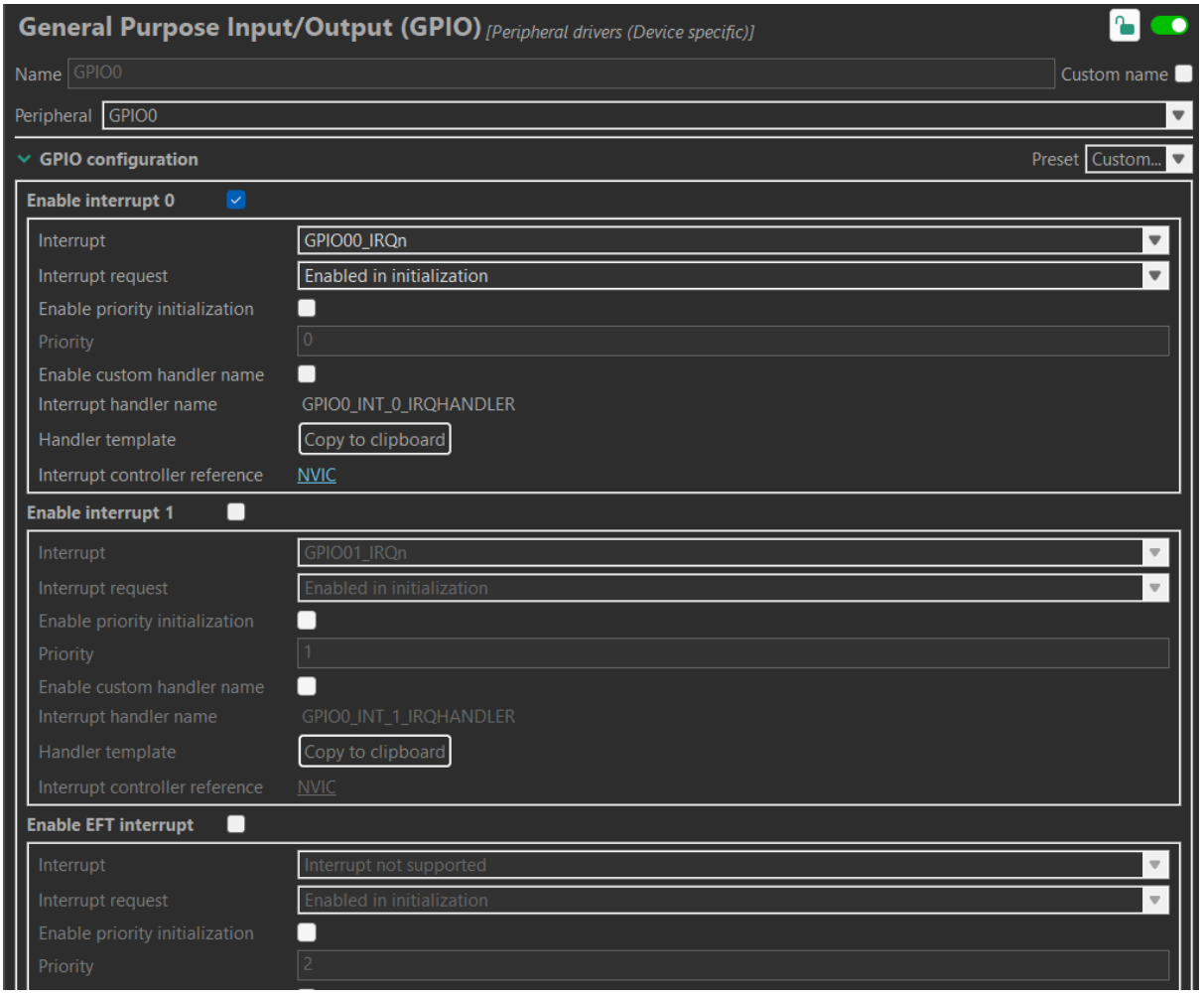
VREF:

El módulo VREF0 genera una referencia de voltaje estable para otros periféricos analógicos como el ADC o el DAC. Está configurado en modo de alto rendimiento (High-power buffer enabled), lo que mejora la precisión y estabilidad de la señal de referencia. La inicialización del ajuste fino (trim) está deshabilitada. Esta configuración prioriza precisión y estabilidad sobre el bajo consumo.



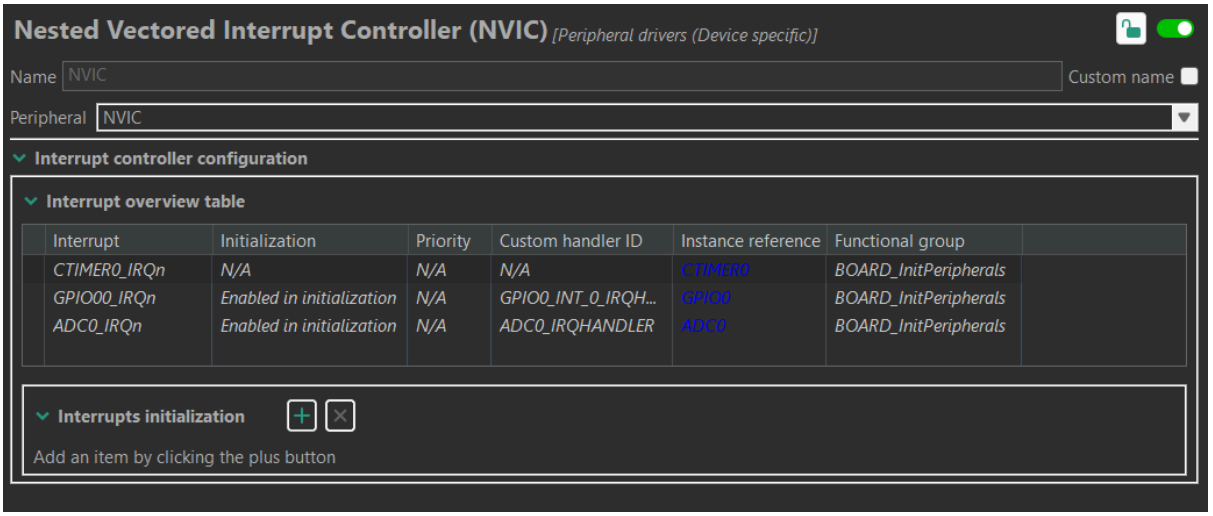
GPIO:

El módulo GPIO0 (General Purpose Input/Output) está configurado con la interrupción 0 habilitada (por eventos en el puerto 0), utilizando el vector GPIO00_IRQn el cual está vinculado al controlador de interrupciones NVIC.



NVIC:

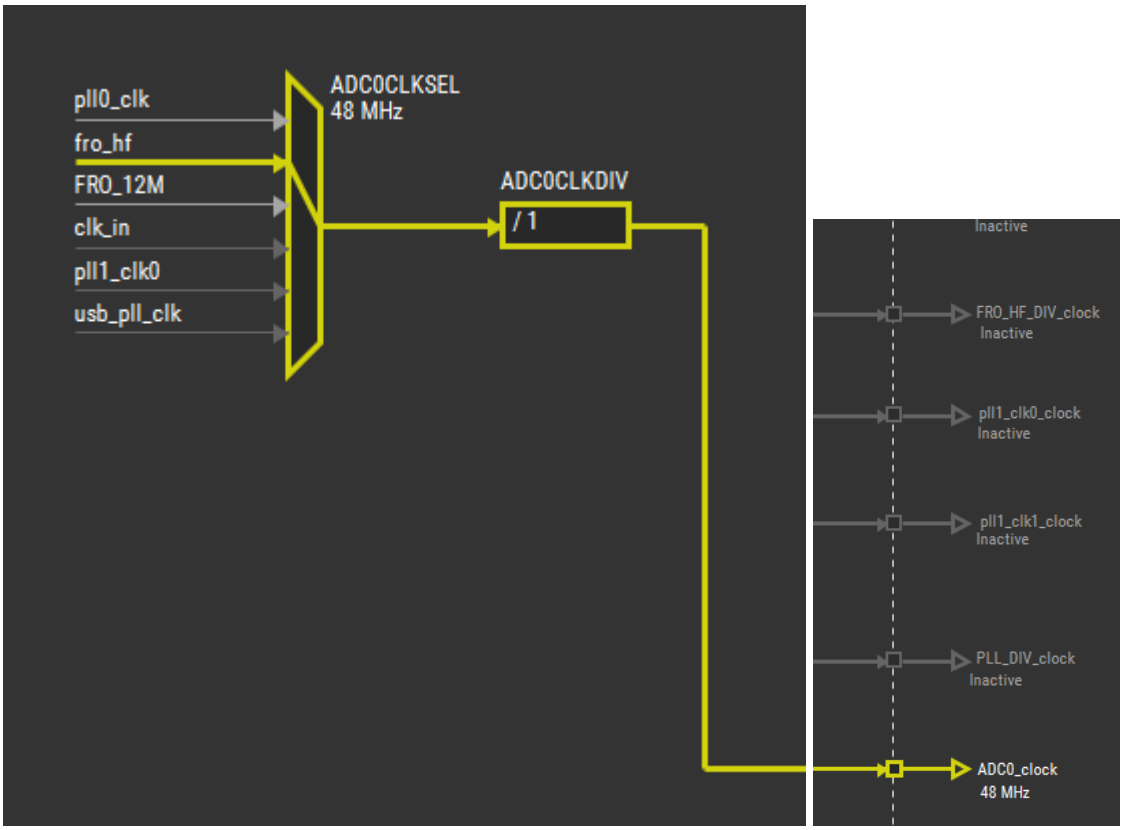
El módulo NVIC (Nested Vectored Interrupt Controller) gestiona las interrupciones del sistema. En esta configuración, se tienen registradas tres interrupciones: CTIMER0_IRQn, GPIO00_IRQn. Esta última asociada al GPIO0_INT_0_IRQHandler. Además la request ADC0_IRQn manejada por el ADC0_IRQHandler. Todas forman parte del grupo funcional BOARD_InitPeripherals, lo que indica que se activan al inicializar los periféricos de la placa.



CLOCKS

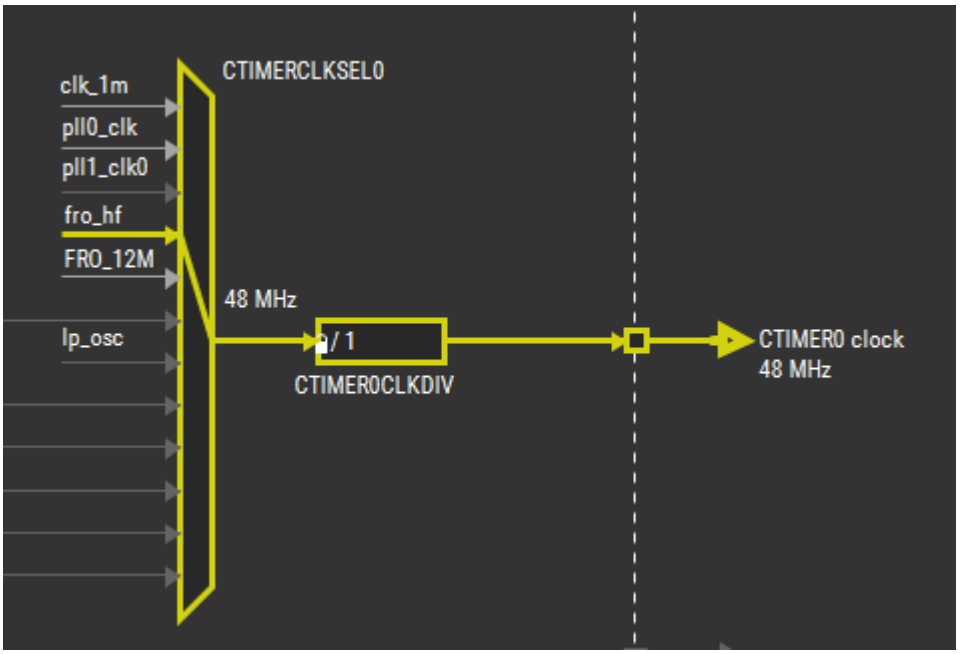
Se se activan los clocks para el funcionamiento de los periféricos.

ADC:



DAC:

TIMER0:



Código del sistema:

Includes de Headers:

Esta sección del programa incluye los archivos de cabecera (headers) necesarios para que en el programa se puedan implementar las distintas funciones y periféricos del microcontrolador. Los headers necesarios para desarrollar el software son los siguientes:

fsl_debug_console.h: permite usar funciones de depuración y consola como PRINTF para mostrar mensajes por UART. Se implemental para monitorear variables o eventos durante la ejecución.

board.h: define la configuración específica de la placa de desarrollo (en este caso MCXN947), como pines de leds, botones, osciladores, relojes y otras características hardware que varían según el modelo de placa.

fsl_ctimer.h: proporciona funciones para manejar los temporizadores generales del microcontrolador, permitiendo iniciar, detener, configurar match, capture y generar interrupciones temporizadas. En nuestro caso implementaremos CTIMER y MATCH en conjunto.

fsl_dac.h incluye funciones para manejar el convertidor digital-analógico, permitiendo enviar valores digitales y generar señales analógicas.

fsl_lpadc.h incluye funciones para manejar el ADC de baja potencia, permitiendo leer valores analógicos desde pines, configurar triggers, interrupciones y resolución de conversión.

fsl_vref.h permite configurar la referencia de voltaje interna usada por el ADC o DAC para mantener mediciones precisas y consistentes.

fsl_inputmux.h y fsl_inputmux_connections.h permiten configurar el multiplexor de entradas del microcontrolador, por ejemplo, para que un timer genere un trigger para el ADC.

peripherals.h es un archivo propio del proyecto que contiene la configuración y prototipos de todos los periféricos que se van a usar, normalmente generado por las herramientas de NXP.

pin_mux.h es un archivo generado por las herramientas de NXP que contiene la configuración de los pines físicos, asignando funciones como GPIO, UART, ADC o DAC a los pines de la placa.

```
/*
    ******************************************************************************
 * Includes
 ******************************************************************************
    */

#include  fsl_debug_console.h
#include  board.h
#include  app.h
#include  fsl_ctimer.h
#include  fsl_dac.h  //HACER UPDATE CODE DESDE EL CONGITOOL O AGREGAR LIBRERIA
    EN PROJECT Y MANAGMENT SDK
#include  fsl_lpadc.h
#include  fsl_vref.h
#include  fsl_inputmux.h
#include  fsl_inputmux_connections.h
#include  peripherals.h
#include  pin_mux.h
```

Bloque de definiciones (variables, constantes, array, enum):

Esta bloque de código define constantes, variables y estructuras necesarias para la adquisición de señales con el ADC y el control de la frecuencia de muestreo. Se establece un buffer para almacenar 512 muestras en formato de 16 bits y otro en formato Q15, y se crean variables para guardar la lectura del ADC, la salida del DAC, los pulsadores y un indicador que señala si la adquisición está activa.

Se incluye un índice para recorrer el buffer de manera circular. Además, se definen las opciones de frecuencia de muestreo mediante un enumerado que incluye 8K, 16K, 22K, 44K y 48K, y se asignan los valores de ticks del timer necesarios para generar las interrupciones correspondientes, considerando la relación entre ticks y microsegundos.

```
/*
    ******************************************************************************
 * Definitions
 ******************************************************************************
    */
#define ADC_BUFFER_SIZE 512 //CANTIDAD DE MUESTRAS A GUARDAR DEL ADC
/*
    ******************************************************************************
 * Variables
 ******************************************************************************
    */
uint32_t contador=0;
uint32_t matchNewValue; //PARA ACTUALIZAR EL DESBORDAMIENTO DEL TIMERO
uint8_t SW3;
uint8_t SW2;
uint16_t valueADC = 0; //PARA ALMACENAR LA LECTURA DEL ADC
uint16_t valueDAC = 0; //PARA VER SALIDA DE DAC EN DEBUUGER
static volatile bool runAcq = false;   // RUN/STOP ADQUISICION
uint16_t adc_samples[ADC_BUFFER_SIZE]; //VECTOR QUE ALMACENA LAS MUESTRAS DEL ADC
typedef int16_t q15_t;      // USAREMOS ENTERO CON SIGNO DE 16 BITS 1 BIT PARA EL SIGNO Y 15 PARA LO DEMAS ,PERO LO LLAMAREMOS CON q15_t
q15_t adc_samples_q15[ADC_BUFFER_SIZE]; //VECTOR QUE ALMACENA LAS MUESTRAS DEL ADC EN FORMATO Q15
uint16_t index =0;      //INDICE DEL BUFFER
//GENERAMOS LAS OPCIONES DE FRECUENCIA:
typedef enum { FS_8K, FS_16K, FS_22K, FS_44K, FS_48K, FS_COUNT } fs_t;
static fs_t select_fs = FS_8K;  //INICIALIZO EN 8KHZ
// 1 tick = 1 us
static const uint32_t s_ticksFs[FS_COUNT] = { //LUEGO CAMBIAR AL DOBLE DE FREC POR EL FLANCO
    3000, //TICKS -> 8 kHz  -> 125 us /2  COMO ES POR FLANCO DE BAJADA USARE EL DOBLE DE FRECUENCIA
    1500, //TICKS -> 16 kHz -> 62.5 us /2
    1090, //TICKS -> 22 kHz -> 45.45 us /2
    545, //TICKS -> 44 kHz -> 22.7 us /2
    500  //TICKS -> 48 kHz -> 20.8 us /2
};
```

Bloque de principal del programa:

Este código corresponde a la función principal del programa y se encarga de inicializar y configurar todos los periféricos del sistema (realizados con ConfigTools) antes de comenzar la ejecución. Primero llama a BOARD_InitHardware para configurar los pines y el hardware básico de la placa, y muestra con un led la frecuencia de muestreo inicial.
Luego deshabilita la interrupción del ADC y detiene el temporizador para asegurar que la adquisición esté apagada al inicio. Después se llama a las funciones de configuración de los periféricos GPIO, DAC y ADC, y establece los valores por defecto, configurando la frecuencia inicial del timer y el color del led correspondiente.
Finalmente, obtiene y muestra por consola la frecuencia del reloj del timer0 y entra en el bucle infinito que mantiene el programa en ejecución.

```c
int main(void)
{
    BOARD_InitHardware();//INICIALIZACION DE PINES Y HARDWARE EN GENERAL
    led_ShowFs(select_fs); //COLOR DE LED CORRESPONDIENTE A LA FREC


    //INCIALMENTE NO HAY CONVERSION-> ADQUSICION:OFF
    DisableIRQ(ADC0_IRQn); //DESHABILITO INT NVIC ADC0
    CTIMER_StopTimer(CTIMER0);                                //PARAMOS EL TIMER


    confGPIO();
    confDAC();
    confADC();

    //INICIALIZO VALORES POR DEFECTO:
    led_ShowFs(select_fs); //COLOR DE LED CORRESPONDIENTE A LA FREC POR DEFECTO 8K
    setNewMatch(select_fs);//SETEA LA FREC EN EL MATCH

    //EnableIRQ(CTIMER0_IRQn);


    uint32_t f_clk = CLOCK_GetCTimerClkFreq(0U); //PARA IMPRIMIR LA FREC DE LA FUENTE DE CLOCK DEL TIMER0
    PRINTF( CTIMER0 clock = %d Hz\r\n , f_clk);

    while (1)
    {
    }
    return 0;
}
```

Bloque de definición de funciones:

En este bloque de código se definen los encabezados de las funciones a implementar.

```c
/*
    ******************************************************************************
 * Prototypes
 ******************************************************************************
    */

void confADC();
void confDAC();
void confGPIO();

void led_ShowFs(fs_t selectFS); //SETEA COLOR SEGUN FREC
void setNewMatch(fs_t selectFS); //SETEA FREC DE MATCH SEGUN ENUM
```

Bloque de funciones de color de LED:

Este bloque de código define funciones para controlar los tres leds del sistema RGB de la placa. Cada función escribe un valor lógico en el pin correspondiente al color rojo, verde o azul, usando la función GPIO_PinWrite. El valor escrito depende del parámetro recibido. Si es verdadero se apaga el led y si es falso se enciende, ya que el hardware trabaja con lógica invertida. La función LED_SetRGB permite controlar los tres leds simultáneamente,

llamando a las funciones de cada color. En conjunto, este bloque permite encender o apagar de forma eficiente los leds RGB de la placa.

```
/*
    ******************************************************************************
 * APLICACIONES:
        //NOTA: INLINE OPTIMIZA LLAMADAS CORTAS, ES MAS EFICIENTE
 ******************************************************************************
    */
//RGB - COLORES:
static inline void LED_R(bool on) {
    GPIO_PinWrite(GPIO0, BOARD_INITPINS_LED_RED_GPIO_PIN, on ? 0u : 1u);
}
static inline void LED_G(bool on) {
    GPIO_PinWrite(GPIO0, BOARD_INITPINS_LED_GREEN_GPIO_PIN, on ? 0u : 1u);
}
static inline void LED_B(bool on) {
    GPIO_PinWrite(GPIO1, BOARD_INITPINS_LED_BLUE_GPIO_PIN, on ? 0u : 1u);
}
static inline void LED_SetRGB(bool r, bool g, bool b) {
    LED_R(r); LED_G(g); LED_B(b);

}
```

Bloques de configuración del sistema:

confADC: Configuración del módulo ADC.

La función configura el ADC para generar una interrupción al completar una conversión y habilita dicha interrupción en el NVIC.

```
void confADC()      //CONFIGURAMOS EL ADC PARA QUE INICIE CONVERSION CON EL
    TRIGGER DEL TIMER0 MATCH3
{
    LPADC_EnableInterrupts(ADC0, kLPADC_Trigger0CompletionInterruptEnable);
    EnableIRQ(ADC0_IRQn); //HABILITO INT NVIC ADC0
}
```

confDAC: Configuración del módulo DAC.

La función habilita el DAC.

```
void confDAC()
{
    DAC_Enable(DAC0, true); //HABILITO DAC0
}
```

confGPIO: Configuración de GPIO.

Se habilita la interrupción del módulo GPIO.

```c
void confGPIO()
{
    EnableIRQ(GPIO00_IRQn);   // HABILITA GPIO00 LA IRQ DE PIO0_6 (SW3)
}
```

Bloques de interrupción del sistema:

CTIMER0_IRQHandler: Rutina de interrupción debida a CTIMER0.

Esta función se implementa para saber durante el proceso de debugging cuando se produce la interrupción.

```c
void  CTIMER0_IRQHandler()
{
    CTIMER_ClearStatusFlags(CTIMER0, kCTIMER_Match3Flag);//LIMPIO FLAG DE TIMER
        0
}
```

GPIO00_IRQHandler: Rutina de interrupción debida a GPIO.

Este handler se ejecuta cuando hay un evento en los pines SW3 (P0.6) o SW2 (P0.23). Al presionar SW3, cambia la frecuencia de muestreo a la siguiente disponible entre 8K, 16K, 22K, 44K y 48K, actualiza el LED correspondiente y ajusta el timer0 con la nueva frecuencia. Al presionar SW2, activa o desactiva la adquisición de datos. Si se activa, configura el timer y el LED según la frecuencia seleccionada, y si se desactiva, detiene el timer. Finalmente, limpia los flags de interrupción de ambos pines para poder detectar futuros eventos.

```c
void GPIO00_IRQHandler(void)  // SE MODIFICA LA FRECUENCIA DE TIMER0
{
    PRINTF( PASO POR INTER GPIO0: \r\n );

    SW3 = GPIO_PinGetInterruptFlag(GPIO0, 6); //FLAG DE INTERRUPCION SI PRESIONE PULSADOR SW3 FLANCO DE SUBIDA
    SW2 = GPIO_PinGetInterruptFlag(GPIO0, 23); //FLAG DE INTERRUPCION SI PRESIONE PULSADOR SW2 FLANCO DE BAJADA

      if(SW3)   // CAMBIO DE VELOCIDAD 8K 16K 22K 44K 48K
      {
            static const char *names[FS_COUNT] = { 8K , 16K , 22K , 44K , 48K };
            select_fs = (select_fs + 1) % FS_COUNT;  // rotamos

            led_ShowFs(select_fs); //COLOR DE LED CORRESPONDIENTE A LA FREC
            setNewMatch(select_fs);//SETEA LA FREC EN EL MATCH
                    // si runAcq=ON arranca ya con la nueva Fs; si OFF queda listo
            PRINTF( Fs -> %s\r\n , names[select_fs]);
      }
      if(SW2)   //ON/OFF ADQUISICION
      {
              runAcq = !runAcq;
              PRINTF( Muestreo %s\r\n , runAcq ?  ON  :  OFF );

              if (runAcq) {
                  setNewMatch(select_fs);//SETEA LA FREC EN EL MATCH  POR DEFECTO LA 8KHZ (ENUM 0)
                  led_ShowFs(select_fs); //SETEA EL COLOR DEL LED POR DEFECTO PARA 8K
              } else {
                  CTIMER_StopTimer(CTIMER0);
              }
      }
    GPIO_PinClearInterruptFlag(GPIO0, 6); //LIMPIO FLAG DE SW3 (GPIO0 PIN 6)
    GPIO_PinClearInterruptFlag(GPIO0, 23); //LIMPIO FLAG DE SW2 (GPIO0 PIN 23)
}
```

ADC0_IRQHandler: Rutina de interrupción del módulo ADC.

Este código es el handler de interrupción del ADC0 y se ejecuta cada vez el Timer Counter es igual al valor del Match. Esta rutina primero lee y limpia los flags de interrupción del ADC para poder detectar futuras conversiones. Luego obtiene el valor convertido del ADC de 16 bits y lo almacena en la variable valueADC. Este valor se envía al DAC de 12 bits desplazando 4 bits hacia la derecha, para ajustar la resolución de 16 bits del ADC a los 12 bits del DAC. Además, guarda la muestra en dos formatos: como uint16_t en adc_samples y como Q15 en adc_samples_q15. Finalmente, incrementa el índice del buffer circular, de manera que las 512 muestras se almacenan de forma cíclica, sobrescribiendo las más antiguas cuando se llena el buffer.

```c
void ADC0_IRQHandler(void) {

  //PRINTF( PASO POR INTER ADC0: \r\n );
  uint32_t trigger_status_flag;
  uint32_t status_flag;
  /* Trigger interrupt flags */
  trigger_status_flag = LPADC_GetTriggerStatusFlags(ADC0);
  /* Interrupt flags */
  status_flag = LPADC_GetStatusFlags(ADC0);
  /* Clears trigger interrupt flags */
  LPADC_ClearTriggerStatusFlags(ADC0, trigger_status_flag);
  /* Clears interrupt flags */
  LPADC_ClearStatusFlags(ADC0, status_flag);

  /* Place your code here */

  lpadc_conv_result_t result;

  /*ADC_16bit A DAC_12bit*/
  LPADC_GetConvResult(ADC0, &result, 0);
  valueADC = result.convValue;
  DAC_SetData(DAC0,valueADC>>4);    //DESPLAZAMOS 4 BITS A LA IZQUIERDA YA QUE LOS SAMPLES DEL ADC SON DE 16BITS, MIENTRAS QUE EL DAC DE 12BITS, POR LO QUE SE DESCARTA

  //GUARDAMOS SAMPLES DEL ADC EN FORMATO Q15, COMO TAMBIEN uint16_t:
  adc_samples[index] = valueADC;                    //GUARDAMOS MUESTRAS DEL ADC EN FORMATO uint16_t
  adc_samples_q15[index] = adc16_to_q15(valueADC);  //GUARDAMOS MUESTRAS DEL ADC EN FORMATO Q15
  index = (index+1)%ADC_BUFFER_SIZE;                //INDICE DE BUFFER CIRCULAR DE 512 MUESTRAS 0,1,2,...,510,511,0,1,2...
}
```

Bloques de códigos funcionales del sistema:

led_ShowFs: Encendido de LEDs en función de la frecuencia de muestreo.

Este código define una función que enciende un LED RGB con un color distinto según la frecuencia de muestreo seleccionada (selectFS). Utiliza un switch para asignar un color específico a cada valor de frecuencia. El LED se apaga si no hay coincidencia.

```c
void led_ShowFs(fs_t selectFS)  //MUESTRA EL COLOR EN EL LED RGB SEGUN LA FREC DE MUESTREO selectFS
{
    switch (selectFS) {
    case FS_8K:  LED_SetRGB(true,  false, false); break; // ROJO
    case FS_16K: LED_SetRGB(false, true,  false); break; // VERDE
    case FS_22K: LED_SetRGB(false, false, true ); break; // AZUL
    case FS_44K: LED_SetRGB(true,  true,  false); break; // AMARILLO
    case FS_48K: LED_SetRGB(false, true,  true ); break; // CIAN
    default:     LED_SetRGB(false, false, false); break; // OFF
    }
}
```

setNewMatch: Cambio de valores de MATCH para interrupción.

Esta función ajusta el valor de coincidencia (match) del temporizador según la frecuencia de muestreo seleccionada (selectFS). En primer instancia detiene el timer y limpia el flag del match 3, luego calcula el nuevo valor de comparación en ticks y lo carga directamente en el registro MR[3]. Si la adquisición (runAcq) está habilitada, reinicia y arranca nuevamente el temporizador y habilita la interrupción del ADC para continuar con la toma de muestras.

```c
void setNewMatch(fs_t selectFS) //CON selectFS SELECCIONA LA FREC DEL ENUM
{
    CTIMER_StopTimer(CTIMER0);                              //PARAMOS EL TIMER
    CTIMER_ClearStatusFlags(CTIMER0, kCTIMER_Match3Flag);   //LIMPIAMOS FLAG PENDIENTE DEL MATCH 3

    uint32_t newMatch = (uint32_t)s_ticksFs[selectFS] - 1U;     //NUEVO MATCH (EN TICKS)

    // CAMBIA DIRECTAMENTE EL REGISTRO MR[3]
    CTIMER0->MR[kCTIMER_Match_3] = newMatch; //CARGAMOS EL NUEVO MATCH (EN TICKS)


    if(runAcq){                 //SOLO SI ESTA HABILITADO  EL INICIO DE ADQUISICION, SINO, CONTINUARA EN STOP EL TIMER
        CTIMER_Reset(CTIMER0);      //REINICIMAOS EL TC DEL TIMER
        CTIMER_StartTimer(CTIMER0); //EMPEZAMOS LA CUENTA NUEVAMENTE
        EnableIRQ(ADC0_IRQn); //HABILITO INT NVIC ADC0

    }
}
```

Bloque de conversión de valor de 16-bits a q15:

El código define una función que convierte un valor de 16 bits obtenido del ADC, que va de 0 a 65535, al formato Q15, que representa números fraccionarios con signo. Primero resta 32768 al valor del ADC para centrarlo en cero, obteniendo un rango de -32768 a +32767, y luego devuelve el resultado como un tipo q15_t.

El formato Q15 es una representación en punto fijo, donde se usa un entero de 16 bits con un bit para el signo y 15 bits para la parte fraccionaria. Por ejemplo, el valor 32767 representa aproximadamente +0.9999 y -32768 representa -1. Este formato se usa en sistemas embebidos porque permite realizar operaciones matemáticas como multiplicaciones y filtros digitales de manera eficiente sin usar punto flotante, lo que ahorra tiempo de procesamiento y recursos en microcontroladores que no tienen unidad de coma flotante.

```c
//CONVERSION  A Q15:
static inline q15_t adc16_to_q15(uint16_t valueADC)
{
    int32_t q15_centrado = (int32_t) valueADC - 32768; // CENTRA EN 0 -> RANGO [-32768, +32767]
    return (q15_t)q15_centrado;     //DEVUELVE EN FORMATO Q15
}
```

6. CONCLUSIONES

Se logró desarrollar un sistema completo de adquisición, procesamiento y reconstrucción de señales analógicas utilizando los periféricos integrados del microcontrolador presente en la placa MCXN947 (ADC, DAC, temporizadores, GPIO y NVIC). A través del diseño e implementación del firmware, se logró digitalizar una señal analógica a distintas velocidades de muestreo (8 kS/s, 16 kS/s, 22 kS/s, 44 kS/s y 48 kS/s), seleccionables mediante una tecla de usuario, con indicación visual del modo activo a través del LED RGB.

El sistema implementa un buffer circular de 512 muestras en formato uint16_t y otro en formato Q15, lo que permite almacenar de forma continua los datos digitalizados y mantener la coherencia en la adquisición. La señal digital obtenida se reconstruye simultáneamente por el DAC de 12 bits, posibilitando verificar en tiempo real la fidelidad entre la señal de entrada y la reconstruida.

Durante el proceso de diseño del sistema se implementaron dos formas de programación. Una de ellas por medio de la herramienta gráfica de configTools y otra por medio de la implementación de drivers para MCXN947.

En la implementación se realizaron modificaciones tales como el disparo de conversión del ADC, que en primer instancia se realizaba en el handler del CTIMER0 y se ejecutaba un disparo del ADC por medio de software lo que producía un retardo en el disparo del ADC. Para corregir este defecto se configuró el disparo del ADC para estar en sincronismo con el match3 del ctimer0.

Para la prueba del funcionamiento del módulo del DAC se escribía la señal de salida con samples de una señal senoidal genérica generada con un script de Python.

La utilización del formato Q15 demuestra la aplicación de técnicas de punto fijo para optimizar el uso de recursos en sistemas sin unidad de coma flotante como es nuestro caso. En el siguiente laboratorio se implementará un filtro digital con diferentes características por lo que se operará con el tipo de dato mencionado anteriormente.